

Bank Customer Churn Prediction — Detailed Explanation

1. Project Title

Bank Customer Churn Prediction Using Machine Learning and Deep Learning

The project predicts whether a bank customer is likely to **leave the bank (churn)** or **stay**.

Example:

Customer information → ML/DL Model → Churn Prediction

2. Abstract

The abstract is a short summary of the entire project.

Your project uses customer information such as:

- Credit score
- Geography
- Gender
- Age
- Tenure
- Account balance
- Number of products
- Credit card status
- Active member status
- Estimated salary

The model learns patterns from historical customer data.

Then, when a new customer's information is entered into the Streamlit application, the model predicts:

Likely to Churn

or

Likely to Stay

The purpose is to help banks identify customers who may be at risk and take appropriate retention actions.

3. Introduction

Banks have a large number of customers.

Some customers continue using banking services, while others may:

- close their account,
- stop using services,
- move to another bank, or
- become inactive.

This is called **customer churn**.

Predicting churn is useful because keeping an existing customer can be more efficient than continuously acquiring new customers.

Your project uses **Data Science and Machine Learning** to analyze historical customer information and identify patterns associated with churn.

4. Problem Statement

The main problem is:

How can we predict whether a bank customer is likely to leave the bank using historical customer data?

For example:

Age = 45
Credit Score = 650
Balance = 120000
Products = 2
Active Member = Yes

The model processes these values and produces a prediction.

Prediction:
LIKELY TO STAY

or

Prediction:
LIKELY TO CHURN

5. Objectives

The main objectives are:

Objective 1 — Data Analysis

Understand customer information and identify patterns.

Objective 2 — Data Preprocessing

Clean the dataset before using it for machine learning.

Objective 3 — Model Training

Train classification models using historical customer data.

Objective 4 — Model Evaluation

Measure model performance using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Objective 5 — Prediction

Predict churn for new customers.

Objective 6 — Deployment

Create a Streamlit web application and make it available online.

6. Dataset

Your project uses a bank customer dataset, referred to in your project as:

European_Bank.csv

The dataset contains customer-related information.

Typical columns include:

Column	Meaning
CreditScore	Customer credit score
Geography	Customer location
Gender	Customer gender
Age	Customer age
Tenure	Years with bank
Balance	Account balance
NumOfProducts	Number of bank products
HasCrCard	Credit card ownership
IsActiveMember	Whether customer is active
EstimatedSalary	Estimated salary
Exited	Churn target

The important target column is generally:

Exited

where:

0 = Customer stayed

1 = Customer exited

You should verify the exact column names in your CSV because they must match your training code.

7. Data Preprocessing

Raw data cannot always be directly given to a machine-learning model.

First, we inspect the data.

Missing values

```
df.isnull().sum()
```

This checks whether any columns contain missing values.

Duplicate records

```
df.duplicated().sum()
```

This checks for duplicate rows.

Separate features and target

Conceptually:

```
X = df.drop("Exited", axis=1)
y = df["Exited"]
```

X contains customer information.

y contains the answer we want the model to learn.

8. Categorical Data

Some values are text.

For example:

Geography
France
Germany
Spain

A machine-learning model normally needs numerical representations.

Therefore, categorical variables are encoded.

For example:

France → 0
Germany → 1
Spain → 2

The exact encoding depends on the preprocessing method used by your training code.

9. Feature Scaling

Some numerical values can have very different ranges.

For example:

Age = 45
CreditScore = 650
Balance = 125000
EstimatedSalary = 85000

Scaling can make numerical features more suitable for algorithms that are sensitive to feature magnitude.

Your project may use a scaler such as:

StandardScaler()

The trained scaler is saved as:

churn_scaler.pkl

This is important because the **same scaler must be used when making predictions**.

10. Exploratory Data Analysis — EDA

EDA means **Exploratory Data Analysis**.

It helps us understand the dataset before training the model.

You can analyze:

Churn distribution

How many customers stayed and how many left?

Age vs churn

Determine whether churn varies across age groups.

Geography vs churn

Compare churn patterns across different locations.

Active membership vs churn

Check whether inactive customers have different churn behavior.

Balance vs churn

Study the relationship between account balance and churn.

Number of products vs churn

Analyze whether customers with different numbers of products behave differently.

EDA is important because it helps identify useful patterns and potential data problems.

11. Machine Learning

This is the main predictive part of the project.

Because the target has two outcomes:

0 = Stay

1 = Churn

this is a **binary classification problem**.

Possible algorithms include:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine
- K-Nearest Neighbors
- Naive Bayes

Your project can compare multiple models and select an appropriate model based on evaluation results.

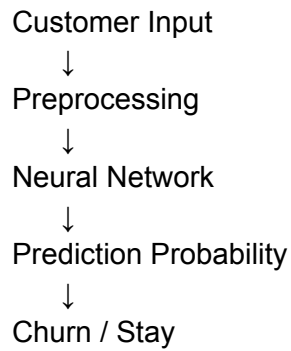
12. Deep Learning

Your project also has a deep-learning configuration using TensorFlow/Keras.

The saved model is:

bank_churn_deep_learning.keras

The general flow is:



The `.keras` file contains the trained neural-network model.

13. Train/Test Split

The dataset is divided into two parts.

Training data

Used to teach the model.

Testing data

Used to evaluate how well the trained model performs on previously unseen records.

For example:

80% → Training
20% → Testing

The exact split used should match your `train_model.py`.

14. Model Evaluation

After training, the model needs to be tested.

Accuracy

Measures the overall percentage of correct predictions.

Example:

Accuracy = 90%

means approximately 90 out of 100 test predictions were correct.

However, accuracy alone is not enough.

Precision

Precision answers:

Of the customers predicted as churners, how many actually churned?

Recall

Recall answers:

Of all actual churners, how many did the model successfully identify?

F1-score

F1-score combines precision and recall.

Confusion Matrix

It shows:

		Predicted	
		Stay	Churn
Actual Stay		TN	FP
Actual Churn		FN	TP

This helps you understand exactly where the model is making mistakes.

Do not put invented accuracy numbers in your final report. Use the actual output from your trained model.

15. Model Saving

After training, you don't want to retrain the model every time somebody opens your website.

Therefore, the trained model is saved.

Your project structure can contain:

```
models/  
|
```

- bank_churn_deep_learning.keras
- churn_scaler.pkl
- feature_columns.pkl

bank_churn_deep_learning.keras

Stores the trained neural-network model.

churn_scaler.pkl

Stores the preprocessing/scaling object.

feature_columns.pkl

Stores the expected feature structure/order.

This is extremely important for prediction.

16. Streamlit Application

Your **app.py** is the web application.

The user opens the website and enters customer information.

For example:

Credit Score: 650
Age: 40
Balance: 100000
Number of Products: 2
Active Member: Yes

Then the application sends the information to the preprocessing pipeline and model.

The flow is:

User
↓
Streamlit UI
↓
Customer Input
↓
Preprocessing
↓
Saved Model

↓
Prediction
↓
Result

17. Prediction Result

The application can display a clear result such as:

 **Customer Likely to Stay**

or

 **Customer Likely to Churn**

You can also display prediction probability if your model supports it.

For example:

Churn Probability: 72%

Be careful to describe this as a **model probability/score**, not a guarantee.

18. Your UI/UX Pages

Based on the interface you have been building, you can organize the application as:

-  Home
-  Customer Review
-  ML Prediction
-  Analytics

Customer Review

Displays customer-related information.

ML Prediction

Allows the user to enter customer details and get a prediction.

Analytics

Displays charts and important dataset insights.

This makes your application look more professional than a simple prediction form.

19. Project Folder Structure

Your project can look like:

Bank Customer Churn/

```
|
|— app.py
|— train_model.py
|— European_Bank.csv
|— requirements.txt
|— Dockerfile
|— README.md
|
|— models/
    |— bank_churn_deep_learning.keras
    |— churn_scaler.pkl
    |— feature_columns.pkl
```

app.py

Runs the Streamlit website.

train_model.py

Trains and saves the model.

European_Bank.csv

Contains the dataset.

requirements.txt

Contains Python dependencies.

Dockerfile

Used for containerized deployment.

README.md

Explains the project on GitHub.

models/

Contains trained model and preprocessing files.

20. Requirements

Your `requirements.txt` might contain packages such as:

```
streamlit  
pandas  
numpy  
scikit-learn  
tensorflow  
joblib  
matplotlib  
seaborn
```

The exact list should match the imports used by your final project.

21. Run the Project Locally

Open PowerShell:

```
cd "C:\UnifiedMentor\Bank Customer Churn"
```

Create virtual environment:

```
python -m venv venv
```

Activate:

```
venv\Scripts\activate
```

Install packages:

```
pip install -r requirements.txt
```

Run:

```
streamlit run app.py
```

Then open the Streamlit address shown in the terminal.

22. Upload to GitHub

After testing the application:

```
git init
git add .
git commit -m "Bank Customer Churn Prediction project"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Your project is then stored online in your GitHub repository.

23. Deploy Online

After GitHub upload, use Streamlit Community Cloud.

Deployment configuration:

Repository: Bank-Customer-Churn-Prediction

Branch: main

Main file: app.py

Then click **Deploy**.

Your application gets an online Streamlit address.

The complete process is:

Python Project



Test Locally



Git



GitHub



Streamlit Community Cloud



Online Website

24. Limitations

Your report should also honestly mention limitations.

For example:

- The prediction depends on the quality of the training dataset.
 - Historical patterns may not always represent future customer behavior.
 - The model can make incorrect predictions.
 - Dataset imbalance can affect evaluation.
 - Real banking systems require stronger security and privacy controls.
 - The model should not be the only factor used for real customer decisions.
-

25. Future Enhancements

You can improve the project by adding:

Explainable AI

Use SHAP to explain why the model predicts churn.

Better Dashboard

Add:

- churn percentage,
- customer statistics,
- interactive charts,
- geography analysis.

Database

Connect the application to MySQL or PostgreSQL.

Authentication

Add login functionality for authorized users.

Model Monitoring

Track model performance after deployment.

Automatic Retraining

Periodically retrain the model using new customer data.

26. Conclusion

The final conclusion can be:

The Bank Customer Churn Prediction project demonstrates the practical application of data science, machine learning, deep learning, and web deployment to a real-world banking problem. The system processes customer information, applies appropriate preprocessing techniques, and uses a trained predictive model to estimate whether a customer is likely to churn.

The project also demonstrates an end-to-end workflow from dataset analysis and model development to Streamlit application development and online deployment. The resulting application provides a simple interface through which users can enter customer information and receive a model-based prediction.

Although the system is primarily developed as a data science and portfolio project, it provides a foundation for future improvements such as explainable AI, advanced analytics, database integration, model monitoring, authentication, and automated retraining.